

DROS & Vajra DSL: The Execution Trust Contract Standard for Heterogeneous AI Systems

1. 商業痛點與戰略定位 (Business Pain Points & Strategic Positioning)

在現代企業的 AI 部署中，自主 Agent 隨處湧現。然而，一旦 Agent 被提示詞劫持或失控，它們可能在無人類指紋的情況下，執行未授權的敏感操作（如轉帳、修改數據）。這帶來了嚴峻的監管與合規赤字（如對接 EU AI Act）。

當前資安治理面臨根本性的「上下文丟失問題 (Context Loss Problem)」：1. 內核空間 (SELinux/eBPF)：雖然安全，但對應用層處於「Context-Blind (上下文盲區)」，只知進程 PID，無法區分具體 Agent 角色與會話。2. 應用空間 (Middleware/Hooks)：雖然具備上下文，但容易被開發者直接呼叫 tool 繞過，防護脆弱。

DROS 的戰略定位：DROS 定義了 AI 時代的 **Execution Non-Repudiation** (執行不可否認性) 標準。我們卡在 **Tool Invocation** OS System Call 的邊界 (即 **FFI/C-ABI 咽喉處**)，既能保留 Agent 上下文，又能保證底層攔截的物理硬度。DROS 提供開源憑證標準 (policy.bin)，藉此推動行業標準的網路效應 (Network Effect)。

2. 架構物理學：FFI 邊界檢查哨 (Architectural Physics & FFI Boundary)

在系統架構中，當 AI 試圖觸碰實體世界時，其指令最終都必須通過語言運行時與系統調用的邊界。DROS 選擇在此 FFI/C-ABI 咽喉處建立一個 Local-first、零延遲的檢核站。

```
graph TD
    subgraph UserSpace ["使用者空間 (User Space)"]
        AI["AI Agents (Python, LangChain, AutoGen)"] --> Token["Token Generation & Speculation"]
    end
    subgraph Gateway ["DROS 通關閘口 (C-ABI/FFI)"]
        Token --> |Tool Call & Context| Interceptor["O(1) Bitwise AND Interceptor"]
    end
    subgraph KernelSpace ["作業系統內核 (Kernel Space)"]
        Interceptor --> |Allow| OS["OS System Calls (Disk, Network, Process)"]
        Interceptor --> |Deny| HardwareException["0.37ms Fail-Closed Hardware Exception (Panic)"]
    end

    style UserSpace fill:#1e1e24,stroke:#333,stroke-width:2px,color:#fff
    style Gateway fill:#2a2b36,stroke:#ff9f1c,stroke-width:3px,color:#fff
    style KernelSpace fill:#111216,stroke:#e71d36,stroke-width:2px,color:#fff
```

在作業系統的世界被嚴格劃分為使用者空間 (User Space) 與內核空間 (Kernel Space) 的物理定律下，DROS 將防禦核心實體建置在「通關閘口的檢核站」上。系統採用 Go/Zig 等底層語言編譯為二進位動態連結庫 (如 .dll / .so / .aar)，透過 C-FFI 或 JNI 暴露給高階語言。DROS 不去使用者空間干涉 AI 生成的 Token，也不修改作業系統內核。我們直接在 **C-ABI 咽喉處進行 O(1) 確定性攔截**。

3. 信任模型與編譯期治理規範 (Trust Model & Compile-Time Governance)

3.0 Agentic Web 公鑰基礎設施與根憑證發行 (Root CA & PKI for Agentic Web)

本架構為自主代理網路 (Agentic Web) 首創之公鑰基礎設施 (PKI) 與根憑證發行機制 (Root CA)。透過將合規規則在編譯期展開並簽名為二進位憑證 (`policy.bin`)。DROS 賦予 AI 攜帶式能力憑證 (Capability Certificate)。

此二進位憑證具備緊湊的物理格式佈局：1. **Header & Version (7 Bytes)**: 固定標頭 VAJRAC + 0x03。2. **Asymmetric CA Signature (64 Bytes)**: Ed25519 密碼學簽章，保護其餘數據免遭篡改。3. **Metadata & Offsets (33 Bytes)**: 含唯一憑證 UUID (16 Bytes)、發行/到期時間戳、以及各區段 (Agent, Tool, Matrix) 的二進位文件偏移量。4. **Lexicographical Indexes**: 按字典順序排序並以 Null 結尾的 Agent ID 與 Tool Name 尋址表。5. **Capability Bitmap Matrix**: 緊密包裝的 bit 矩陣，用於執行期常數時間 $\mathcal{O}(1)$ 權限比對。

接收端的 **VajraClaw** 檢核站直接以 Root CA 公鑰進行奈秒級無狀態離線驗證，徹底防堵無痕 AGI 代理在跨網路調用中的作惡空間，並提供無可抵賴的執行法律責任歸屬。

3.1 宣告式合約 (Policy-as-Code) 與嚴格相容

企業透過 Vajra DSL 宣告式語言來定義複雜的公司治理與權限。* Vajra 規範在根節點強制要求宣告 `vajra_version: 1`，缺失此欄位將導致致命編譯錯誤 (FATAL compile error)，確保向下相容性。* 系統遵循「預設拒絕 (Deny-by-default)」的安全原則：缺失策略、未知 Agent 或未知 Tool 皆預設為 DENY。在規則衝突時，DENY 永遠優先於 ALLOW (DENY > ALLOW)。

3.2 決定性編譯 (Deterministic Compilation) 與企業級門禁

- **數學一致性**：編譯器在產生 `policy.bin` 時會強制執行嚴格的排序規則 (Agents, Tools, Capabilities)。只要傳入相同的私鑰與策略內容，產出的二進位簽章與 SHA-256 Hash 將永遠絕對一致。
- **健康檢查與阻斷**：透過 `vajra doctor`，CISO 能直接總覽策略複雜度、未使用的 Capabilities 與稀疏矩陣 (Sparse Matrix) 風險，避免 IAM 爆炸。若 `vajra lint` 掃描出 [ERROR] 或 [CRITICAL] (例如非管理員 Agent 被授予了 `admin.*` 全域萬用字元)，編譯將立即中斷 (Abort)，物理性阻斷不安全設定上線。

3.3 $\mathcal{O}(1)$ 執行期決定性 ($\mathcal{O}(1)$ Runtime Determinism)

執行期絕對不進行任何字串或 JSON 解析，僅執行純粹的位元運算 (Bitwise AND)。這確保了零延遲與完全的決定性 (Determinism)，徹底根絕 TOCTOU (Time-of-Check to Time-of-Use) 風險。一旦偵測到越權，微內核將觸發硬體級 Exception (Panic)，實現 **0.37 毫秒的物理熔斷與 Fail-Closed 防禦**。

4. 企業安全邊界與稽核一致性 (Enterprise Scale & Audit Consistency)

4.1 密碼學稽核日誌 (Cryptographic Audit Binding)

傳統日誌無法證明決策依據。DROS 創新性地將執行的二進位策略檔之 **SHA-256 Policy Hash** 與 **Compiler Version** 強制注入至每一筆 Audit JSON 中。這為企業提供了具備不可否認性 (Non-repudiation) 的「策略指紋」鐵證。

4.2 零信任權限分離與降級模式 (Zero-Trust Privilege Separation)

- **OS 級實體防護**：在 Kubernetes/Docker 部署中，策略檔作為 Read-Only ConfigMap 掛載。AI Agent 絕對無法被授予寫入權限，從 OS 層面物理性拒絕竄改。
- **離線降級 (Mode C)**：在行動端邊緣裝置 (Gomobile Android AAR) 缺乏網路時，引擎自動降級為嚴格唯讀模式，物理阻斷所有寫入行為 (Offline Safe Degraded)。

4.3 部署規模極限 (Scale Limits & Mesh OTA)

- **Enterprise Audit 規模限制**：基於專用伺服器硬體資源與網卡物理平衡，標準企業版限制為單機 30 個並行 Agents，綁定 15 個 Machine UUIDs，總計支援 **450 個並行 Agents**。
- **VajraAgent Mesh 控制平面**：超大規模部署可引導至 Mesh 架構，支援 fully On-Premise 實體隔離 (Air-Gapped) 部署、K8s/Ansible 自動化，以及具備 1ms 全域緊急斷路廣播控制權的 Mesh OTA。

5. 實作範例：Vajra DSL 執行合約 (Execution Contract Spec)

以下展示 Vajra DSL 如何將商業邏輯與權限抽象化，並在編譯期轉譯為 O(1) 攔截依據：

```
# =====  
# DROS Vajra Policy - Dynamic Agentic Execution Contract  
# =====  
vajra_version: 1  
  
agents:  
- id: aws_langchain_agent  
  role: general_support  
- id: local_autogen_admin  
  role: system_admin  
  
capabilities:  
  aws_langchain_agent:  
    - READ_KNOWLEDGE_BASE  
  local_autogen_admin:  
    - READ_KNOWLEDGE_BASE  
    - WRITE_SYSTEM_LOGS
```

```

    - EXECUTE_REBOOT

tools:
  # 基於能力的安全映射 (Capability-Based Security)
  - name: internal_db.read.*
    requires:
      - READ_KNOWLEDGE_BASE
  - name: os.system.reboot
    requires:
      - EXECUTE_REBOOT

rules:
  # 絕對防線：任何嘗試刪除日誌的行為，優先級最高 (DENY > ALLOW)
  - match:
      agent: "*"
      tool: os.system.delete_logs
      effect: DENY

  # 明確授權：編譯期將自動展開 Wildcard，鑄造為 O(1) Bitmap
  - match:
      agent: aws_langchain_agent
      tool: internal_db.read.customer_info
      effect: ALLOW

```